

Roll no: **43221**

Title: **Assignment 2: Implementing Feedforward neural networks with Keras and TensorFlow**

- a. Import the necessary packages
- b. Load the training and testing data (CIFAR10)
- c. Define the network architecture using Keras
- d. Train the model using SGD
- e. Evaluate the network
- f. Plot the training loss and accuracy

Datasets: CIFAR10

a. Import the necessary packages

```
In [ ]: import tensorflow as tf
        from tensorflow import keras
        from tensorflow.keras import layers
        from tensorflow.keras.layers import Input, Conv2D, Dense, Flatten, Dropout
        from tensorflow.keras.layers import GlobalMaxPooling2D, MaxPooling2D
        from tensorflow.keras.layers import BatchNormalization
        from tensorflow.keras.preprocessing.image import ImageDataGenerator
        from tensorflow.keras.models import Model
        from keras.utils import to_categorical
        import matplotlib.pyplot as plt
        import numpy as np
```

b. Load the training and testing data (MNIST/CIFAR10)

Note: Decrease batch size and remove some layers if epoch takes too much time to run

```
In [ ]: train_data_dir = 'Datasets/cifar-10-img/train'
        test_data_dir = 'Datasets/cifar-10-img/test'

        # Image data generator for training data
        train_datagen = ImageDataGenerator(
            rescale=1.0/255
        )

        # Image data generator for testing data
        test_datagen = ImageDataGenerator(
            rescale=1.0/255
        )

        # Create data generators
        train_batch_size = 20000
        train_generator = train_datagen.flow_from_directory(
            train_data_dir,
            target_size=(32, 32), # Resize images to 32x32
            batch_size=train_batch_size,
            class_mode='sparse',
            shuffle=True,
        )

        # Load test data without labels (class_mode=None)
        test_batch_size = 1000
        test_generator = test_datagen.flow_from_directory(
            test_data_dir,
            target_size=(32, 32), # Resize images to 32x32
            batch_size=test_batch_size,
            class_mode='sparse',
            shuffle=True,
        )
```

Found 40079 images belonging to 10 classes.

Found 40079 images belonging to 10 classes.

Selecting first batch containing 20000 images

```
In [ ]: x_train, y_train = train_generator[0]
        x_test, y_test = test_generator[0]
```

```
In [ ]: y_train = y_train.flatten()
        y_test = y_test.flatten()
```

c. Define the network architecture using Keras

Note: remove some layers if epoch takes to much time to run

```
In [ ]: # number of classes
K = len(set(y_train))
# calculate total number of classes
# for output layer
print("number of classes:", K)
i = Input(shape=x_train[0].shape)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(i)
x = BatchNormalization()(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2))(x)

x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2))(x)

x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
x = BatchNormalization()(x)
x = MaxPooling2D((2, 2))(x)

x = Flatten()(x)
x = Dropout(0.2)(x)

# Hidden layer
x = Dense(1024, activation='relu')(x)
x = Dropout(0.2)(x)

# last hidden layer i.e.. output layer
x = Dense(K, activation='softmax')(x)

model = Model(i, x)
```

number of classes: 10

d. Train the model using SGD

```
In [ ]: model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy',  
metrics=['accuracy'])  
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test,  
y_test))
```

Epoch 1/10

2023-11-03 18:12:38.995176: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 245760000
exceeds 10% of free system memory.

625/625 [=====] - 52s 81ms/step - loss: 1.7270 -
accuracy: 0.4166 - val_loss: 1.3471 - val_accuracy: 0.5140

Epoch 2/10

625/625 [=====] - 52s 84ms/step - loss: 1.2254 -
accuracy: 0.5589 - val_loss: 1.0913 - val_accuracy: 0.5960

Epoch 3/10

625/625 [=====] - 54s 86ms/step - loss: 1.0257 -
accuracy: 0.6321 - val_loss: 0.9400 - val_accuracy: 0.6720

Epoch 4/10

625/625 [=====] - 52s 83ms/step - loss: 0.9049 -
accuracy: 0.6770 - val_loss: 1.0236 - val_accuracy: 0.6460

Epoch 5/10

625/625 [=====] - 52s 83ms/step - loss: 0.7892 -
accuracy: 0.7211 - val_loss: 0.8201 - val_accuracy: 0.7210

Epoch 6/10

625/625 [=====] - 52s 84ms/step - loss: 0.7015 -
accuracy: 0.7531 - val_loss: 0.7621 - val_accuracy: 0.7300

Epoch 7/10

625/625 [=====] - 54s 86ms/step - loss: 0.6171 -
accuracy: 0.7807 - val_loss: 0.6825 - val_accuracy: 0.7720

Epoch 8/10

625/625 [=====] - 53s 84ms/step - loss: 0.5426 -
accuracy: 0.8073 - val_loss: 0.6558 - val_accuracy: 0.7860

Epoch 9/10

625/625 [=====] - 52s 83ms/step - loss: 0.4794 -
accuracy: 0.8322 - val_loss: 0.6498 - val_accuracy: 0.7810

Epoch 10/10

625/625 [=====] - 53s 84ms/step - loss: 0.4111 -
accuracy: 0.8555 - val_loss: 0.5968 - val_accuracy: 0.8260

e. Evaluate the network

```
In [ ]: test_loss, test_acc = model.evaluate(x_test, y_test)  
print("Loss: ", test_loss)  
print("Accuracy: ", test_acc)
```

32/32 [=====] - 0s 14ms/step - loss: 0.5968 -
accuracy: 0.8260

Loss: 0.5967786312103271

Accuracy: 0.8259999752044678

```
In [ ]: import matplotlib.pyplot as plt
        y = to_categorical(y_test)
        predicted_value = model.predict(x_test)
```

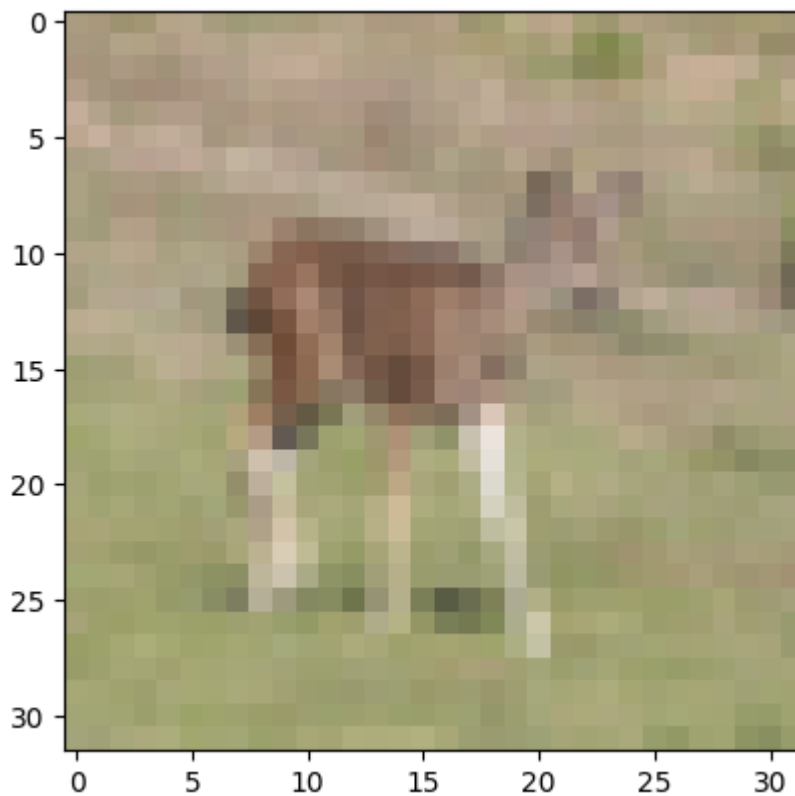
32/32 [=====] - 1s 13ms/step

```
In [ ]: labels = list(test_generator.class_indices.keys())
```

Change n to test for other images

```
In [ ]: n = 10
        plt.imshow(x_test[n])
        print("Actual Number: ", labels[np.argmax(y[n])])
        print("Predicted Number: ", labels[np.argmax(predicted_value[n])])
```

Actual Number: deer
Predicted Number: deer



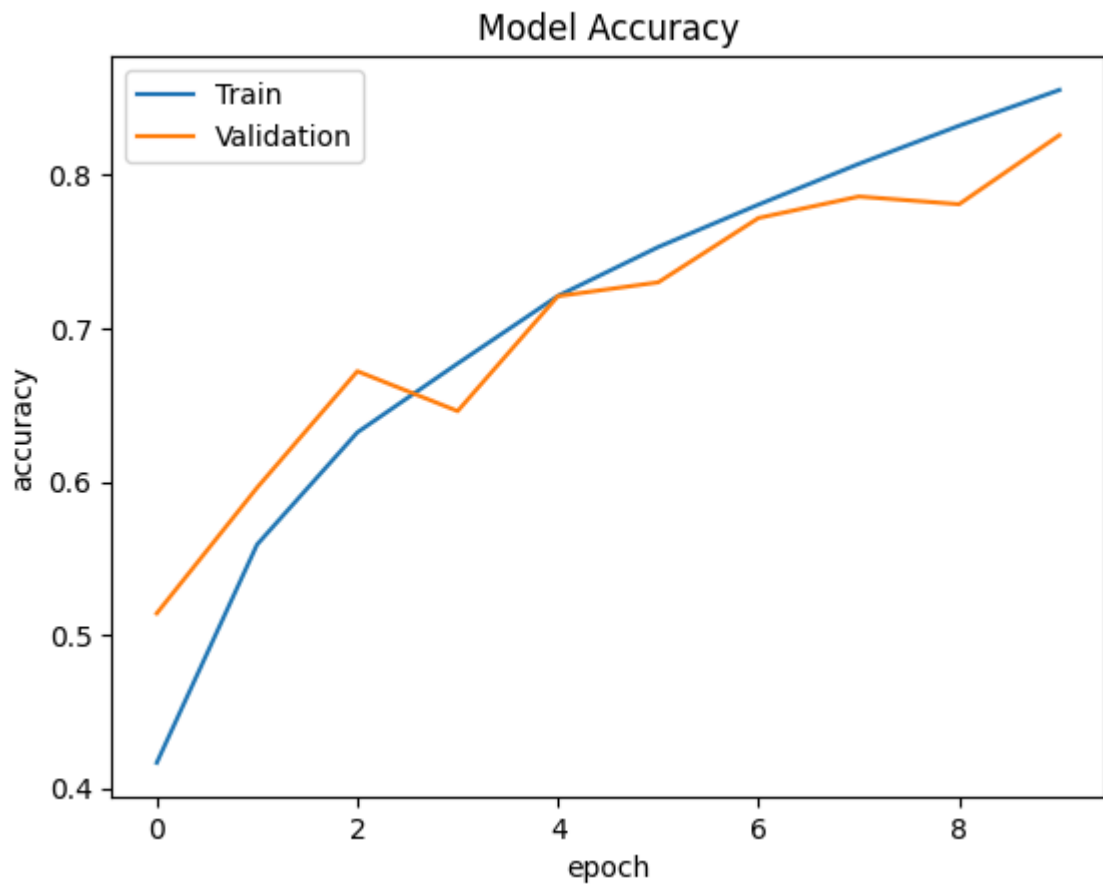
f. Plot the training loss and accuracy

```
In [ ]: history = history.history
        history.keys()
```

```
Out[53]: dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

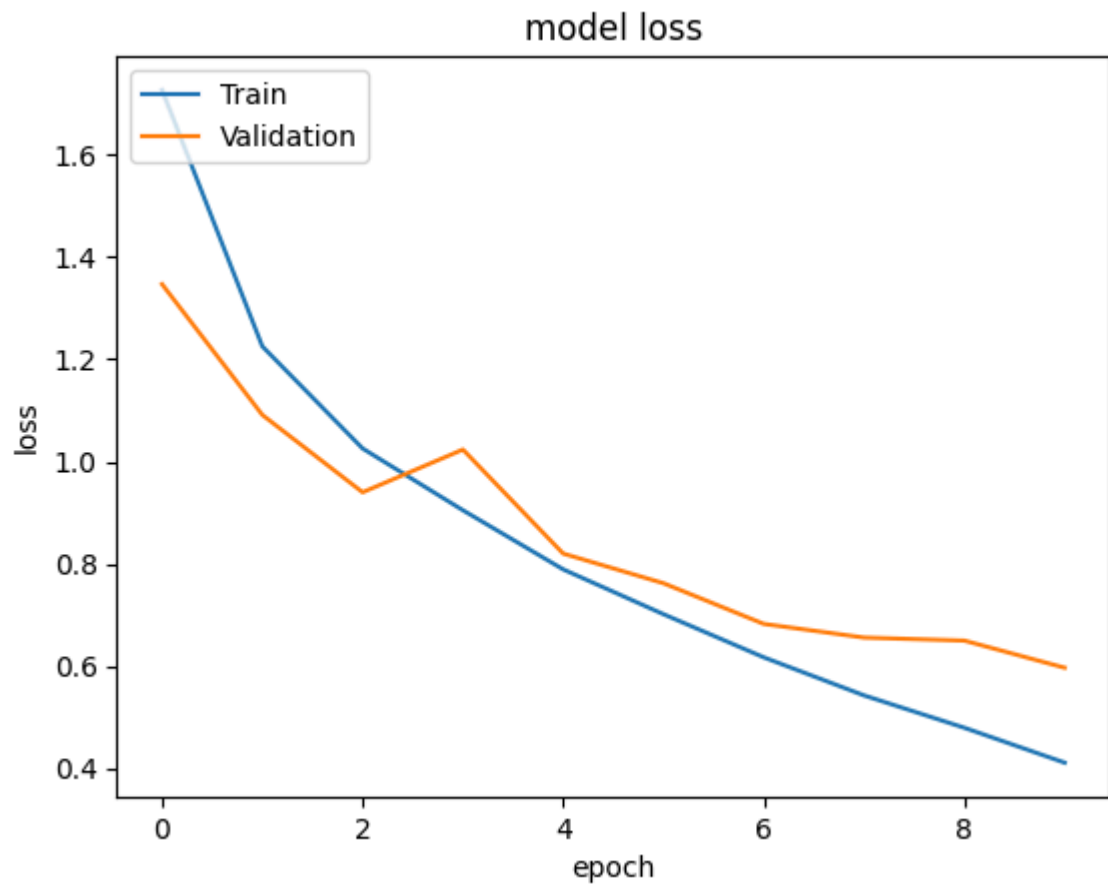
```
In [ ]: plt.plot(history['accuracy'])
plt.plot(history['val_accuracy'])
plt.title("Model Accuracy")
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['Train', "Validation"], loc='upper left')
```

Out[54]: <matplotlib.legend.Legend at 0x7fc4e464c3d0>



```
In [ ]: plt.plot(history['loss'])
plt.plot(history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
```

Out[55]: <matplotlib.legend.Legend at 0x7fc4f5dff2d0>



```
In [ ]: plt.plot(history['accuracy'])
plt.plot(history['val_accuracy'])
plt.plot(history['loss'])
plt.plot(history['val_loss'])
plt.title('Training Loss and accuracy')
plt.ylabel('accuracy/Loss')
plt.xlabel('epoch')
plt.legend(['accuracy', 'val_accuracy', 'loss', 'val_loss'])
```

Out[15]: <matplotlib.legend.Legend at 0x7f463c2ad3d0>

